



National University
Of Computer and Emerging Sciences

Semester End Project

In Partial Fulfillment of the requirement
for the Course of

Design and Analysis of Algorithms
CS-2009

By:

INAM ULLAH SHAIKH 22I-0857 H

ABDUL MUNHIM HUSSAIN 22I-1021 H

Table of Contents

Question 1	3
Pseudo Code:.....	3
Time Complexity Analysis:	3
Question 2	4
Pseudo Code:.....	4
Description of Sub-problems Space:.....	5
Recurrence Relation:	5
Bottom-up implementation of dynamic programming:	5
Time Complexity Analysis:	6
Space complexity Analysis:	6
Question 3	7
Pseudo Code:.....	7
Time Complexity Analysis:	8
Question 4	9
Pseudo Code:.....	9
Time Complexity Analysis:	10

Part A:

Question 1

Pseudo Code:

FUNCTION calculateStructures(n)

 for i = 0 to n

 for j = 0 to n

 if i = 0

 dp[i][j] = 1

 else

 dp[i][j] = 0

 for i = 1 to n

 for j = 1 to n

 if j <= i

 dp[i][j] = dp[i][j - 1] + dp[i - j][j - 1]

 else

 dp[i][j] = dp[i][j - 1]

 return dp[n][n - 1]

Time Complexity Analysis:

Initialization of dp array: There are two nested loops to initialize the dp array with dimensions $(n+1) \times (n+1)$, where n is the input parameter.

Time Complexity: $O((n + 1)^2)$, as it involves visiting each cell of the array once.

Dynamic Programming Update: Again, there are two nested loops, each running from 1 to n, iterating over the dp array. Within these loops, there are constant-time operations.

The total number of iterations for both loops combined is $O((n + 1)^2)$.

Inside the inner loop, there are constant-time operations as well.

Therefore, the time complexity of the dynamic programming update step is $O(n^2)$.

Overall Time Complexity:

Since the dominant factor in terms of time complexity is the dynamic programming update step, which is $O(n^2)$, the overall time complexity of the **calculateStructures** function is $O(n^2)$.

Question 2

Pseudo Code:

FUNCTION initializeMemo(size, attacks):

 memo = allocate 2D array of size [size][attacks]

 for i = 0 to size - 1:

 for j = 0 to attacks - 1:

 memo[i][j] = -1

FUNCTION initialize(arr, DP, cols):

 for i = 0 to cols - 1:

 for j = 0 to cols - 1:

 DP[i][j] = 0

 prevj = 0

 while prevj < cols:

 j = prevj + 1

 i = 0

 while j < cols:

 DP[i][j] = DP[i][j - 1] + DP[i + 1][j] - DP[i + 1][j - 1] + arr[i] * arr[j]

 i = i + 1

 j = j + 1

 prevj = prevj + 1

FUNCTION BestStrategy(DP, start, end, noofattacks):

 if end - start <= noofattacks:

 return 0

 if noofattacks == 0:

 return DP[start][end]

 if memo[start][noofattacks] != -1:

 return memo[start][noofattacks]

 result = INT_MAX

```

for split = 0 to end - 1:
    right = BestStrategy(DP, split + 1, end, noofattacks - 1)
    left = DP[start][split]
    result = min(result, left + right)
memo[start][noofattacks] = result
return result

```

Description of Sub-problems Space:

For this question our main problem is the total array and our sub-problems are the smaller combinations made from that array

In our implementation, the sub-problems are represented by the memoization array `memo[start][noofattacks]`, where `start` indicates the starting index of the segment of the railroad, and `noofattacks` represents the number of attacks remaining for that specific starting index.

The overall goal is reflected in these sub-problems by considering different segments of the railroad and the number of attacks available, aiming to find the minimum strategic value that Abdullah can achieve.

Recurrence Relation:

$$\text{BestStrategy}(start, end, attack) = \begin{cases} 0, & end - start = attack \\ 0, & start = end \\ DP[start][end], & attack = 0 \\ memo[start][attack], & memo[start][attack] \neq -1 \\ \min(start \leq i < end (BestStrategy(i + 1, end, attack - 1) + DP[start][i]) \end{cases}$$

Bottom-up implementation of dynamic programming:

Start with the smallest sub-problems, which correspond to single depots or depot segments of length 1.

Gradually increase the size of the segments, computing the strategic value for each possible segment length.

Use the computed values for smaller segments to compute the values for larger segments, ensuring that all necessary dependencies are available, we are ensuring that we don't calculate already calculated values, we just store them in **memoisation table** and use them when needed.

Continue this process until we compute the strategic value for the entire railroad segment with the given number of attacks available.

Specifically, we would iterate over all possible segment lengths and depot configurations, starting from segments of length 1 and incrementally increasing the length until we cover the entire railroad segment. This ensures that we have the values for all smaller segments available when computing the values for larger segments.

Time Complexity Analysis:

DP Initialization: The initialize function initializes the DP array with a nested loop over the depots, resulting in $O(n^2)$ operations, where n is the number of depots.

Recurrence Relation Calculation: The BestStrategy function calculates the values of sub-problems using a recursive approach with memoization. In the worst case, it iterates over all possible splits, which can be up to n . Additionally, it iterates over all possible numbers of attacks, which can be up to m . Therefore, the total time complexity for calculating the sub-problems is $O(n * m)$.

Total Time Complexity: Combining the above steps, the overall time complexity of the algorithm is $O(n^2 * m)$,

Space complexity Analysis:

DP Array: The space complexity for the DP array is $O(n^2)$, as it stores the Strategic Values for all possible combinations of depots.

Memoization Array: The memoization array stores the results of sub-problems, and its size is determined by the number of depots and the maximum number of attacks. Therefore, its space complexity is $O(n * m)$.

Additional Space: The algorithm uses additional space for storing linked list nodes and other variables, but these have a negligible impact compared to the DP and memoization arrays.

Overall, the space complexity of the algorithm is dominated by the DP array and memoization array, resulting in $O(n^2 + n * m)$ space complexity, which simplifies to $O(n^2)$.

PART B:

Question 3

Pseudo Code:

```
FUNCTION findmax(arr[], start, end)
```

```
    m = arr[start]
```

```
    FOR i = start + 1 TO end
```

```
        IF arr[i] > m THEN
```

```
            m = arr[i]
```

```
    RETURN m
```

```
FUNCTION firstMissingPos(A[], start, end)
```

```
    nm = findmax(A, start, end) + 2
```

```
    BOOL present[nm] = {false}
```

```
    FOR i = start TO end
```

```
        present[A[i]] = true
```

```
    min = 0
```

```
    WHILE present[min]
```

```
        min++
```

```
    RETURN min
```

```
FUNCTION makesegments(arr[], size)
```

```
    MNPN = firstMissingPos(arr, 0, size - 1)
```

```
    maxsize = findmax(arr, 0, size - 1) + 2
```

```
    BOOL present[maxsize] = {false}
```

```
    right = MNPN
```

```
    left = 0
```

```
    i = 0
```

```
    check = false
```

```
    WHILE i < size
```

```
        IF present[arr[i]] == false THEN
```

```
            present[arr[i]] = true
```

```
            left++
```

```

        IF arr[i] > MNPN THEN
            right++
        IF left == right THEN
            check = true
            BREAK
        i++
    IF check THEN
        IF i == size - 1 THEN
            IF firstMissingPos(arr, 0, i - 1) == firstMissingPos(arr, i, size - 1) THEN
                PRINT "Segments created successfully MNPN = ", MNPN
                PRINT "Segment 1 : 1 - ", i
                PRINT "Segment 2 : ", size, " - ", size
                RETURN
            ELSE
                IF firstMissingPos(arr, 0, i) == firstMissingPos(arr, i + 1, size - 1) THEN
                    PRINT "Segments created successfully MNPN = ", MNPN
                    PRINT "Segment 1 : 1 - ", i + 1
                    PRINT "Segment 2 : ", i + 2, " - ", size
                    RETURN
                PRINT "No segments created -1"

```

Time Complexity Analysis:

1. findmax(arr[], start, end):

This function goes through the array from start to end to find the maximum element.

Time complexity: **O(n)**, where n is the number of elements in the array from start to end.

2. firstMissingPos(A[], start, end):

This function calls findmax once and then iterates through the array A from start to end to find the first missing positive integer.

Time complexity: **O(n)**, where n is the number of elements in the array from start to end.

3. makesegments(arr[], size):

This function calls firstMissingPos and findmax once each, which takes **O(n)** time in total. Then it iterates through the array once, which also takes **O(n)** time in the worst case. Inside the

loop, it checks if the left and right segments have the same number of unique elements. This comparison also takes **O(1)** time.

So, the overall time complexity of this function is O(n).

Question 4

Pseudo Code:

```
FUNCTION MaxPower(Arr, rows, cols)
```

```
    if rows == 1
```

```
        max = 0
```

```
        for i = 0 to cols
```

```
            if Arr[i] > max
```

```
                max = Arr[i]
```

```
        return max
```

```
// initialize SumArr as a 2D array of size rows x cols, filled with zeros
```

```
for i = 0 to rows - 1
```

```
    SumArr[i][0] = Arr[i][0]
```

```
for i = 0 to cols - 1
```

```
    max[0] = 0
```

```
    indexes[0] = 0
```

```
    max[1] = 0
```

```
    indexes[1] = 0
```

```
    for j = 0 to rows - 1
```

```
        if SumArr[j][i] > max[0]
```

```
            max[0] = SumArr[j][i]
```

```
            indexes[0] = j
```

```
    for j = 0 to rows - 1
```

```
        if SumArr[j][i] > max[1] and j != indexes[0]
```

```
            max[1] = SumArr[j][i]
```

```
            indexes[1] = j
```

```
for k = i + 1 to min(cols - 1, i + 2)
```

```
    for l = 0 to rows - 1
```

```
        if l != indexes[0] and max[0] + Arr[l][k] > SumArr[l][k]
```

```

        SumArr[l][k] = max[0] + Arr[l][k]
    for k = i + 1 to min(cols - 1, i + 2)
        for l = 0 to rows - 1
            if l != indexes[1] and max[1] + Arr[l][k] > SumArr[l][k]
                SumArr[l][k] = max[1] + Arr[l][k]
    max = 0
    for i = 0 to rows - 1
        if SumArr[i][cols - 1] > max
            max = SumArr[i][cols - 1]
    return max

```

Time Complexity Analysis:

Initialization of SumArr: This step involves initializing a 2D array SumArr with dimensions rows × cols, setting all elements to zero. Since it involves visiting each element once

Time complexity: $O(\text{rows} * \text{cols})$.

Updating SumArr: There are nested loops iterating over each cell of SumArr to update its value based on the previous cells. The outer loop iterates over the columns (cols iterations), and the inner loop iterates over the rows (rows iterations). Within these loops, there are additional nested loops iterating over a maximum of 3 neighbouring cells each, contributing only a constant factor to the time complexity.

Time complexity: $O(\text{rows} * \text{cols})$.

Finding the maximum value in the last column of SumArr: This step involves a single loop iterating over the rows to find the maximum value in the last column of SumArr. Since it iterates through each row once.

Time complexity is $O(\text{rows})$.

Combining these steps, the dominant factor in the time complexity is the updating of SumArr, which is **$O(\text{rows} * \text{cols})$** .

So, the overall time complexity of this function is $O(\text{rows} * \text{columns})$ which is $O(n * m)$ / $O(n^2)$.